

GTU ROBOTICS CLUB

30 FPS FORMULA

Introduction

This documentation outlines the process of identifying, verifying, and configuring NVIDIA GPU drivers on a modern Linux-based system for optimal performance in AI, machine learning, and GPU-accelerated workloads. The target hardware is a Lenovo IdeaPad Gaming 3 equipped with an 11th Gen Intel Core i5-11320H processor and an NVIDIA GeForce RTX 3050 Laptop GPU (GA107M architecture), operating under Ubuntu 22.04 LTS (amd64 architecture).

Proper driver installation is a critical prerequisite for enabling CUDA, TensorRT, and other GPU-accelerated frameworks both in native Linux environments and inside Docker containers. Without the correct driver, tools such as `nvidia-smi` cannot interface with the GPU, rendering it unusable for compute-intensive tasks.

This guide demonstrates a systematic approach — beginning with hardware detection using commands like `lspci`, `uname`, and `lscpu`, followed by architecture verification, driver compatibility determination, and final driver deployment. The methodology ensures precise driver selection by distinguishing between desktop (non-notebook) and notebook variants of the RTX 30 series, as well as accurately identifying non-Ti versus Ti models.

By the end of this setup, the system will be fully equipped with the latest recommended NVIDIA driver, complete CUDA compatibility, and optional NVIDIA Docker toolkit integration — enabling high-performance GPU computing for advanced AI workflows.

Some Details About This Device

Model: Lenovo IdeaPad Gaming 3 – 15IHU6

Operating System: Ubuntu 22.04 LTS (64-bit, amd64 architecture)

Processor:

- 11th Gen Intel® Core™ i5-11320H
- 4 cores / 8 threads
- Base clock: **3.20 GHz**, Boost clock: up to **4.50 GHz**
- Supports advanced instruction sets including SSE, AVX, AVX2, and AVX-512
- Virtualization support: Intel VT-x

Graphics Processing Unit (GPU):

- NVIDIA **GeForce RTX 3050 Laptop GPU** (GA107M architecture)
- **Notebook** variant, **non-Ti** version
- Dedicated video memory: (e.g., 4 GB GDDR6 – check exact from *nvidia-smi* once driver installed)
- **CUDA**, **Tensor cores**, and Ray Tracing cores supported

Architecture:

- CPU & OS: **x86_64** (amd64)
- Byte Order: Little Endian
- Physical Address size: 39 bits, Virtual Address size: 48 bits

Special Features for AI Workloads:

- Compatible with **NVIDIA CUDA**, **cuDNN**, **TensorRT**, and PyTorch/TensorFlow GPU acceleration
- Can run **GPU** workloads inside Docker via NVIDIA Container Toolkit
- Suitable for training and inference of deep learning models, computer vision pipelines, and real-time robotics workloads

1. Solve the Problem of Nvidia-smi Command

Problem

Running:

```
nvidia-smi
```

returned:

```
Command 'nvidia-smi' not found
```

This indicates the NVIDIA driver was not installed or not detected.

Solution Steps:

1. Verify System Architecture

```
uname -m
```

Output should be:

```
x86_64
```

Confirming this is an AMD64/Intel64 architecture (not ARM).

2. Identify GPU

```
lspci | grep -i nvidia
```

Example output:

```
02:00.0 3D controller: NVIDIA Corporation GA107M  
[GeForce RTX 3050 Mobile] (rev a1)
```

3. Download NVIDIA Driver

Download the correct `.run` driver for your GPU from:
Official Website: <https://www.nvidia.com/en-us/drivers/>

Example file:

[`NVIDIA-Linux-x86\_64-580.76.05.run`](#)

4. Switch to Non-GUI (TTY) Mode

To stop the desktop environment and avoid conflicts:Run this command

`sudo systemctl isolate multi-user.target`

Log in with your Ubuntu username and password in the TTY terminal.

5. Make the Driver Installer Executable

Run this commands:

1.`cd ~/Downloads`

2.`chmod +x NVIDIA-Linux-x86_64-580.76.05.run`

6. Run the Installer

Run this command : `sudo ./NVIDIA-Linux-x86_64-580.76.05.run`

Accept license

Allow DKMS to register the kernel module

Disable Nouveau driver if prompted (system will handle blacklisting)

7. Reboot to GUI

Run this command : **sudo reboot**

8. Verify Installation

Run this command: **nvidia-smi**

Expected output example:

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
NVIDIA-SMI 580.76.05				Driver Version: 580.76.05				CUDA Version: 13.0					
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
GPU Name				Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC			
Fan Temp Perf				Pwr:Usage/Cap				Memory-Usage		GPU-Util Compute M. MIG M.			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
0 NVIDIA GeForce RTX 3050		...		Off		00000000:02:00.0		Off		N/A			
N/A 50C P8				7W / 30W		14MiB / 4096MiB				0% Default N/A			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
Processes:													
GPU GI CI				PID Type		Process name				GPU Memory			
		ID ID										Usage	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
0 N/A N/A				1542 G		/usr/lib/xorg/Xorg				4MiB			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													

That means your driver supports up to CUDA 13.0.
You can install any CUDA toolkit $\leq$ 13.0.

2. Installing and Verifying CUDA Toolkit (nvcc)

1. Check the Maximum Supported CUDA Version

Run:

```
nvvidia-smi
```

Example output:

```
CUDA Version: 13.0
```

This means the installed **NVIDIA driver** supports up to CUDA 13.0.
You must install a **CUDA Toolkit** version less than or equal to this number.

2. Choose the CUDA Toolkit Version

- **CUDA 13.0** → Latest features, good if building modern CUDA software.
- **CUDA 12.x** → More compatible with stable releases of PyTorch/TensorFlow.
- If you use only **prebuilt AI/ML packages**, CUDA Toolkit installation is optional (packages ship with their own runtime).

3. Install CUDA Toolkit

Example for **Ubuntu 22.04** (adjust for your OS version):

Follow this simple steps:

a) Search Cuda Toolkit On Web browser

b) Click Link: <https://developer.nvidia.com/cuda-downloads>

c) Select OS: Linux

Architecture : x86_64

(Run this command for architecture : **uname -m**)

Distribution : Ubuntu

Version: 22.04 (choose your OS Version)

Installer type: deb(local)

Run this all command step by step :

```
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_64/cuda-ubuntu2204.pin
sudo mv cuda-ubuntu2204.pin /etc/apt/preferences.d/cuda-repository-pin-600
wget https://developer.download.nvidia.com/compute/cuda/13.0.0/local_installers/cuda-repo-ubuntu2204-13-0-local_13.0.0-580.65.06-1_amd64.de

sudo dpkg -i cuda-repo-ubuntu2204-13-0-local_13.0.0-580.65.06-1_amd64.deb
sudo cp /var/cuda-repo-ubuntu2204-13-0-local/cuda-*-keyring.gpg /usr/share/keyrings/
sudo apt-get update
sudo apt-get -y install cuda-toolkit-13-0
```

4. Add CUDA to PATH

After above process do this process:

Run this both command in terminal:

1.export PATH=/usr/local/cuda-13.0/bin:\$PATH

2.export LD_LIBRARY_PATH=/usr/local/cuda-13.0/lib64:\$LD_LIBRARY_PATH

Then reload:

3.source ~/.bashrc

5. Testing

Run this command :

nvcc --version

Expected Output:

```
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6:~$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Jul_16_07:30:01_PM_PDT_2025
Cuda compilation tools, release 13.0, V13.0.48
Build cuda_13.0.r13.0/compiler.36260728_0
```


3. Setup of cuDNN :

a) Download and show the all the detail which is compatible with your current cuda version:

(For cuda version check: Run this command
`nvcc --version`)

b) Check all the detail from this link:

Link: <https://developer.nvidia.com/cudnn>
<https://docs.nvidia.com/deeplearning/cudnn/latest/>

Also know which is the compatible version of cuDNN with your current cuda version

c) Click the link

Link: <https://developer.nvidia.com/cudnn-downloads>

Operating System	Linux Windows
Architecture	x86_64 arm64-sbsa aarch64-jetson
Distribution	Tarball Debian OpenSUSE RHEL Rocky SLES Ubuntu
Version	22.04 24.04
Installer Type	deb (local) deb (network)
Configuration	FULL JIT

This is the specification of this device. You can check your device details.

d) Run all the command (this is for cuda version 13.0)

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cudnn/9.12.0/local_installers/cudnn-local-repo-ubuntu2204-9.12.0_1.0-1_amd64.deb
$ sudo dpkg -i cudnn-local-repo-ubuntu2204-9.12.0_1.0-1_amd64.deb
$ sudo cp /var/cudnn-local-repo-ubuntu2204-9.12.0/cudnn-*.keyring.gpg /usr/share/keyrings/
$ sudo apt-get update
$ sudo apt-get -y install cudnn
```

e) Final check:

Run this command: **dpkg -l | grep cudnn**

Expected Output:

```
ttam@purushottam-IdeaPad-Gaming-3-15IHU6:~$ dpkg -l | grep cudnn
ii          9.12.0-1          amd64        NVIDIA CUDA Deep Neural Network library (cuDNN)
ii cudnn-local-repo-ubuntu2204-9.12.0 1.0-1      amd64        cudnn-local repository configuration files
ii          9.12.0-1          amd64        NVIDIA CUDA Deep Neural Network library (cuDNN)
ii nn9-cuda-13      9.12.0.46-1 amd64        NVIDIA cuDNN for CUDA 13
ii nn9-cuda-13-0    9.12.0.46-1 amd64        NVIDIA cuDNN for CUDA 13.0
ii cudnn9-cuda-13   9.12.0.46-1 amd64        cuDNN runtime libraries for CUDA 13.0
ii cudnn9-dev-cuda-13 9.12.0.46-1 amd64        cuDNN development libraries for CUDA 13.0
ii cudnn9-headers-cuda-13 9.12.0.46-1 amd64        cuDNN header files for CUDA 13.0
ii cudnn9-samples   9.12.0.46-1 all          cuDNN samples
ii cudnn9-static-libraries-cuda-13 9.12.0.46-1 amd64        cuDNN static libraries for CUDA 13.0
```

4) Setup of TensorRT Framework

a) check the all the details

which TensorRT version compatible with your current cuda version.

Link : <https://developer.nvidia.com/tensorrt> Go through this website

b) Click Download Now

Select the version which is compatible with cuda version

Note: TensorRT 10.13.2 suitable for CUDA 13.0

So for this system select **TensorRT 10**

Agree the terms

TensorRT 10.13.2 GA for x86_64 Architecture

Debian, RPM, and TAR Install Packages for Linux

- > TensorRT 10.13.2 GA for Linux x86_64 and CUDA 12.0 to 12.9 TAR Package
- > TensorRT 10.13.2 GA for Ubuntu 22.04 and CUDA 12.0 to 12.9 DEB local repo Package
- > TensorRT 10.13.2 GA for Ubuntu 24.04 and CUDA 12.0 to 12.9 DEB local repo Package
- > TensorRT 10.13.2 GA for RedHat / Rocky Linux 8 and CUDA 12.0 to 12.9 RPM local repo Package
- > TensorRT 10.13.2 GA for RedHat / Rocky Linux 9 and CUDA 12.0 to 12.9 RPM local repo Package
- > TensorRT 10.13.2 GA for Linux x86_64 and CUDA 13.0 TAR Package
- > TensorRT 10.13.2 GA for Ubuntu 22.04 and CUDA 13.0 DEB local repo Package
- > TensorRT 10.13.2 GA for Ubuntu 24.04 and CUDA 13.0 DEB local repo Package
- > TensorRT 10.13.2 GA for RedHat / Rocky Linux 8 and CUDA 13.0 RPM local repo Package
- > TensorRT 10.13.2 GA for RedHat / Rocky Linux 9 and CUDA 13.0 RPM local repo Package

This are the option which is appear after agreed the terms.

Select the right package which is suitable for your current system architecture.

For this system we chose :

- TensorRT 10.13.2 GA for Ubuntu 22.04 and CUDA 13.0 DEB local repo Package

1. INSTALL IT : Run the commad

sudo dpkg -i
nv-tensorrt-local-repo-ubuntu2204-10.13.2-cuda-13.0_1.0-1_amd64.deb

2. Add repo key:

sudo cp
/var/nv-tensorrt-local-repo-ubuntu2204-10.13.2-cuda-13.0/nv-tens
orrt-local-AF54650E-keyring.gpg /usr/share/keyrings/

3. Update APT Sources

sudo apt-get update

After this, the repository will be recognized without
key-related errors.

4. Install TensorRT and Optional Tools

sudo apt-get install tensorrt

sudo apt-get install libnvinfer-dev

sudo apt-get install python3-libnvinfer-dev # Python API
support

5. Verify the Installation

dpkg -l | grep nvinfer

Expected Output :

```
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6: $ dpkg -l | grep nvInfer
ii  libnvInfer-bin          10.13.2.6-1+cuda13.0      amd64      TensorRT binaries
ii  libnvInfer-dev          10.13.2.6-1+cuda13.0      amd64      TensorRT development libraries
ii  libnvInfer-dispatch-dev 10.13.2.6-1+cuda13.0      amd64      TensorRT development dispatch runtime libraries
ii  libnvInfer-dispatch10   10.13.2.6-1+cuda13.0      amd64      TensorRT dispatch runtime library
ii  libnvInfer-headers-dev  10.13.2.6-1+cuda13.0      amd64      TensorRT development headers
ii  libnvInfer-headers-plugin-dev 10.13.2.6-1+cuda13.0      amd64      TensorRT plugin headers
ii  libnvInfer-headers-python-plugin-dev 10.13.2.6-1+cuda13.0      amd64      TensorRT Python plugin development headers
ii  libnvInfer-lean-dev     10.13.2.6-1+cuda13.0      amd64      TensorRT lean runtime libraries
ii  libnvInfer-lean10       10.13.2.6-1+cuda13.0      amd64      TensorRT lean runtime library
ii  libnvInfer-plugin-dev   10.13.2.6-1+cuda13.0      amd64      TensorRT plugin libraries
ii  libnvInfer-plugin10     10.13.2.6-1+cuda13.0      amd64      TensorRT plugin libraries
ii  libnvInfer-samples      10.13.2.6-1+cuda13.0      all        TensorRT samples
ii  libnvInfer-vc-plugin-dev 10.13.2.6-1+cuda13.0      amd64      TensorRT vc-plugin library
ii  libnvInfer-vc-plugin10  10.13.2.6-1+cuda13.0      amd64      TensorRT vc-plugin library
ii  libnvInfer-wln-builder-resource10 10.13.2.6-1+cuda13.0      amd64      TensorRT Windows builder resource library
ii  libnvInfer10            10.13.2.6-1+cuda13.0      amd64      TensorRT runtime libraries
ii  python3-libnvInfer      10.13.2.6-1+cuda13.0      amd64      Python 3 bindings for TensorRT standard runtime
ii  python3-libnvInfer-dev  10.13.2.6-1+cuda13.0      amd64      Python 3 development package for TensorRT standard runtime
ii  python3-libnvInfer-dispatch 10.13.2.6-1+cuda13.0      amd64      Python 3 bindings for TensorRT dispatch runtime
ii  python3-libnvInfer-lean 10.13.2.6-1+cuda13.0      amd64      Python 3 bindings for TensorRT lean runtime
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6: $ dpkg -l | grep tensorrt
```

Run this command also:

dpkg -l | grep tensorrt

```
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6: $ dpkg -l | grep tensorrt
ii  nv-tensorrt-local-repo-ubuntu2204-10.13.2-cuda-13.0 1.0-1      amd64      nv-tensorrt-local repository configuration files
ii  tensorrt                                              10.13.2.6-1+cuda13.0      amd64      Meta package for TensorRT
purushottam@purushottam-IdeaPad-Gaming-3-15IHU6: $ nvidia-smi
```

👉 Once done, your laptop will be fully set up:

✅ CUDA 13.0

✅ cuDNN 9.12.0

✅ TensorRT 10.13.2

Read the Yolov8 deployment guide pdf then download and install all the necessary library and Run your code at GPU.

